Mobile Preassembly Systems with Cooperative Dual-Arm Manipulation - A Concept for Industrial Applications in the Near Future

Jonas Wittmann
Chair of Applied Mechanics
TUM School of Engineering and Design
Munich Institute of Robotics and Machine Intelligence
Technical University of Munich
jonas.wittmann@tum.de

Mathias Laile
Chair of Materials Handling,
Material Flow, Logistics
TUM School of Engineering and Design
Technical University of Munich
mathias.laile@tum.de

Johannes Rainer
Chair of Applied Mechanics
TUM School of Engineering and Design
Munich Institute of Robotics and Machine Intelligence
Technical University of Munich
johannes.rainer@tum.de

Johannes Fottner
Chair of Materials Handling,
Material Flow, Logistics
TUM School of Engineering and Design
Technical University of Munich
j.fottner@tum.de

Daniel Rixen
Chair of Applied Mechanics
TUM School of Engineering and Design
Munich Institute of Robotics and Machine Intelligence
Technical University of Munich
rixen@tum.de

***Abstract*—In current manufacturing plants, autonomous guided vehicles (AGVs) are used for the transport of material. Our goal is to expand the use of AGVs, enabling them to perform complex handling and preassembly tasks. For a more efficient production, these tasks should be performed during transport. We propose a concept for a mobile dual-arm robot that we develop in a two-year project. Four modules are at the core of our solution: the *Task Planner* controls the high-level process, the *Dual-Arm Planner* enables coordinated and cooperative dual-arm manipulation, the *Vision* module detects objects and the *Navigator* controls the mobile base. Preliminary experiments demonstrate the navigation and manipulation capabilities of our robot.**

***Index Terms*—dual arm manipulation, mobile manipulation, cooperating robots**

I. Introduction

The current trend towards more customized products requires flexible production lines. At the same time, the production volumes should not decrease. To meet this challenge, AGVs are used in many production facilities for a flexible transport of material. However, their capabilities are limited to standardized boxes [1]. To extend the application possibilities of AGVs, we integrate two collaborative robots, one with an end-effector mounted camera, and a mobile base. In addition, the mobile dual-arm system allows the time spent on transport in today's AGVs to be used for preassembly, making production more efficient. Fig. 1 shows the current state of our mobile dual-arm robot. The contributions of our research project are:

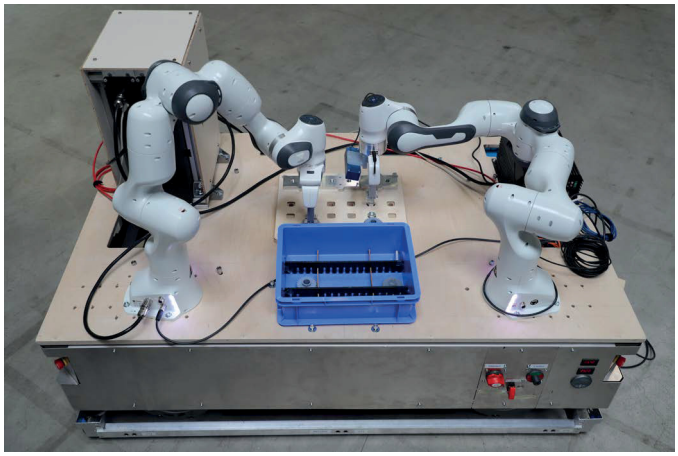


Fig. 1: The hardware of our mobile dual-arm system integrates three components: a custom-built mobile base, two *Franka Emika* robots and the *robominds* vision system. The camera is mounted on the right end-effector.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 08,2026 at 12:57:12 UTC from IEEE Xplore. Restrictions apply.

ISBN: 978-3-8007-6140-1 © VDE VERLAG GMBH · Berlin · Offenbach

- Collision-free path planning in dynamic environments for a single-arm setup has been widely discussed in research. For multi-arm setups, however, this is still an open area and collisions between two robots that share a workspace must be avoided. We contribute robust planning algorithms for a dual-arm setup that exploits efficient collision modeling.
- We use two collaborative robots with integrated torque measurements that allow us to develop algorithms for tactile robotic assembly.
- We use *PDDL 2.1* for adaptive task planning. The task planning framework currently consists of seven *Types*, 24 *State Variables*, and 20 *Actions*. *POPF* [2] is used to compile plans. The task planning is integrated into the software architecture using *ROSPlan* [3].
- When performing manipulation tasks during transport, the motion of the mobile base induces forces on the mounted robot arms. This is not a problem for position-controlled industrial robots, but for collaborative robots that are more sensitive to external disturbances. We develop methods to compensate for these dynamic disturbances in the control of the two manipulators.

This paper presents our system architecture and it puts a special focus on the control of the two mounted robot arms and the methods that enable dual-arm manipulation. Single-arm manipulation is a state-of-the-art automation solution for many industrial processes, e.g. welding. Planning and control algorithms are well studied and solutions have been proposed for different applications like human-robot collaboration or time-optimal planning. There are ready-to-use frameworks that implement a complete manipulation process with a simple user interface: the user or vision system defines a target object and the framework computes collision-free, pregrasping poses and motions to pick the object and place it at a defined target pose. When it comes to dual-arm manipulation, algorithms become more complex and computationally expensive. Full framework solutions are not there yet. There are individual solutions for control systems and planning algorithms. A ready-to-use integration of both, however, is not there. Therefore, dual-arm manipulation is rarely applied in industry, despite its advantages [4]–[6]:

- Increased payload and handling of bigger objects.
- Assembly of two parts without using special fixture.
- Handling of flexible objects.
- Solving complex tasks, e.g. opening a bottle or unscrewing a container.
- Automate work places that were originally designed for humans (human centric environment), e.g. with teleoperation or by learning imitation [7].

The remainder of this paper is organized as follows. Section II presents related work. The hardware and software architecture of our system are described in Section III. Section IV briefly outlines the foreseen logic of the *Task Planner*. Section V details the implementation of the *Dual-Arm Planner* that controls the two mounted robot arms. Section VI presents the state-of-the-art path planning algorithm of the mobile base. Conclusions follow in Section VII.

II. Related Work

Current industrial applications of mobile robotics mostly focus on transport. The few systems that, besides transport, integrate manipulation tasks, are limited to the lifting of boxes [1]. Research in mobile robotics focuses on environment perception and machine learning-based approaches for navigation in dynamic environments [8]. Mobile manipulation, however, is not yet widely used in applications and research and is still an open field, especially for dual-arm setups. In 2015, *ABB* presented the *IRB 14000 (YuMi)*, a dual-arm robot, developed for electronics industry. With a payload of only 500 g, its range of applications is limited and there are only few industrial realization: In [9], *YuMi* delivers parts to a press machine and in [10], a mobile version of *YuMi* is used in a hospital for pipetting, centrifuge loading, and transport. However, there is no manipulation during transport. *Baxter* was a collaborative dual-arm robot by *Rethink Robotics*. However, the design of the drive module focused on safety at the expense of precision, so it failed to reach the market and development was stopped [11]. *duAro* by *Kawasaki* integrates two SCARA manipulators, each with four degrees of freedom (DoF). This limits manipulation tasks to the horizontal direction, and cooperative manipulation, i.e. when both robots manipulate the same object, is not covered [12]. The manipulators of *YASKAWA*'s dual-arm robot *MOTOMAN CSDA10F* are similar in design to our system. However, they do not provide an interface for the joint torque measurements, so it is not possible to implement flexible assembly tasks [13]. *NEXTAGE* is a mobile dual-arm robot with a head-mounted camera and cameras in both end-effectors for object localization. However, the mobile base has to be driven manually by a human operator [14]. *ARMAR-6* [15] is a research platform for mobile dual-arm manipulation with the focus on industrial applications and human-robot cooperation. It consists of five components: a dual-arm system, two five-finger hands, a head with integrated sensors, a torso and the mobile base. Integrated strain gauges in the 16 joints allow to implement compliant behavior when working alongside humans. Manipulation tasks during transport are not covered. [16] presents a design concept for *Rollin' Justin*, a mobile dual-arm manipulator, equipped with omnidirectional wheels like ours to increase its manipulability next to shelves. In our work, the dual-arm robot is not derived from a humanoid upper body, but specifically conceived for industrial applications.

III. Concept

Our research contribution consists in a mobile preassembly system. The goal of the research project is the demonstration of the system capabilities for the production process of a gearbox model whose components are shown Fig. 1. Each gearbox consists of seven different components and the components of several gearboxes are provided on a board on the mobile base. The robots have to pick the gearbox parts and put them into the blue small load carrier according to a predefined logic.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 08,2026 at 12:57:12 UTC from IEEE Xplore. Restrictions apply.
ISBN: 978-3-8007-6140-1

© VDE VERLAG GMBH · Berlin · Offenbach

Further, one preassembly operation, i.e. mounting the gear on the drive shaft, has to be performed during transport.

A. Hardware Architecture

Fig. 1 shows our current hardware setup. The mobile base is a custom-built autonomous mobile robot. Its main components are four mecanum wheels to enable omnidirectional movements, two 270° *Lidar* sensors located at two diagonal edges of the robot, a *SICK Flexi Soft CPU* [17] for monitoring safety fields, and an industrial computer for running the *Robot Operating System (ROS)* and the robot's software stack. The mobile robot is powered by two 100 A h lead acid batteries that will be replaced by two 200 A h lithium batteries. The *Franka Emika* robot provides the required features for manipulation in a shared workspace with humans. The safety is guaranteed by torque measurements in each joint and the lightweight structure (moved mass: 12.8 kg) minimizes harm in case of collisions. The joint torque measurements allow to implement force control algorithms for cooperative manipulation, e.g. in assembly tasks. The seven DoFs provide the necessary flexibility for operating the robot inside boxes. The fingers of the end-effectors are specifically designed for the gearbox components, but they may be adapted easily to other applications. The poses of the components in the storage on the mobile base are known, however, the poses of the blue boxes are not known and have to be localized by a vision system. Further, grasped objects may be lost during transport and must be localized. We use *robominds* [18], a ready-to-use vision system with machine learning-based detection of optimal grasping poses of unknown objects. We mount the camera on one of the end-effectors. The software modules run on a real-time capable robot control computer with 64-bit Ubuntu 20.04.

B. Software Architecture

We use *ROS* to handle the communication between our four software modules:

- The *Task Planner*, outlined in Section IV, assembles sequences of *Actions* and executes them to achieve a successful outcome. *State Variables* are updated by the other modules and reflect the current state of the workspace. Even in non-nominal conditions, they allow the *Task Planner* to find a feasible plan.
- The *Dual-Arm Planner*, outlined in Section V, connects to the mounted manipulators and implements the servers for motion planning and execution of the two arms. Further, it controls the end-effectors during grasping.
- The *Navigator*, outlined in Section VI, operates the mobile base.
- *Vision* wraps the skills provided by the *robominds* system into a ROS node. Specifically, it provides a service to detect the blue boxes or lost components and it provides a service to detect the QR codes that we attach to shelves, so that inaccuracies in the mobile base's navigation can be compensated for.

TABLE I: *State Variables* used by the *Task Planner*.

Predicate Name	Related Objects	
in	part	box
target	robot	grasping-object
holding	robot	grasping-object
target	robot	grasping-object
at-pre-grasp	robot	
at-pre-drop	robot	
at-grasp	robot	
at-after-grasp	robot	
at-drop	robot	
at-after-drop	robot	
empty	robot	
ready	robot	
in-goal	grasping-object	

The *Actions* in the *Task Planner* call *Vision*, *Navigator*, and *Dual-Arm Planner* via *ROS* services and reflect the effects of these *Actions* in the *State Variables*.

IV. Task Planner

The *Task Planner* is based on *PDDL 2.1*. The used components of *PDDL* are *typing* for defining objects, *durative-actions* for defining actions that have a duration, and *fluents* for defining numeric values. The *objects* have a hierarchical structure, with the general type object at the top. The two main objects are currently *robot* and *grasping-object*. The next hierarchy level defines whether a robot is located *left* or *right* on the mobile base. The grasping objects are divided into *part* and *box*. The parts are further grouped into gears, shafts, bolts, and bases. Table I gives an overview of the most important *State Variables* defined by the *Task Planner*. There are further *State Variables* defining which actions the two robot arms can perform in parallel. Moreover, a *PDDL fluent* counts the number of parts dropped at their goal position.

Fig. 2 shows the *PDDL Actions* that are currently necessary for a single robot to grasp a part. Furthermore, the *Task Planner* defines *Actions* for detecting grasping poses, *cooperative* movements of the arms, e.g. for moving a box, as well as assembling parts.

The *PDDL* framework is integrated into the software architecture using *ROSPlan* for generating *PDDL* problems, compiling plans, and dispatching the *Actions* to the software modules as *ROS* services. Initially, the current state is defined by the values that the *State Variables* have at the start of the process. Then, a goal state is defined, e.g. the number of parts that should be in the box. The *Task Planner* compiles a plan by running the *POPF* solver, which is then executed. If an action fails or if, due to an unforeseen event, the *State Variables* of a succeeding action are not met, the *Task Planner* cancels the execution, compiles a new plan based on the current state, and restarts execution.

V. Dual-Arm Planner

Due to the advantages mentioned in Section I, and to pave the way for dual-arm applications in industry, we decided on a dual-arm setup for the manipulation module of our system. In literature, dual-arm manipulation is classified the following [4], [6]:

Authorized licensed use limited to: Tsinghua University. Downloaded on March 08,2026 at 12:57:12 UTC from IEEE Xplore. Restrictions apply.

ISBN: 978-3-8007-6140-1 © VDE VERLAG GMBH · Berlin · Offenbach

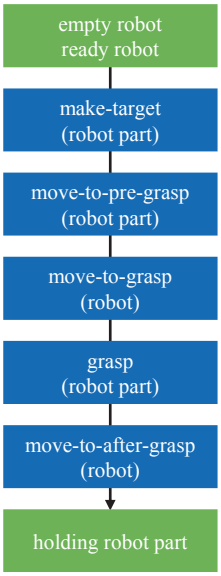


Fig. 2: *PDDL Actions* (blue) and their preconditions/effects (green) to grasp a single part.

- *(Goal-)Coordinated* Manipulation: Both arms manipulate different objects, however, share the same workspace.
- *Cooperative/Bimanual* Manipulation: Both arms manipulate the same object.

Our process given in Section III includes both operations, i.e. a *coordinated* handling of the individual gearbox components, and a *cooperative* handling of the load carrier and a *cooperative* preassembly. This section describes the control design of the two manipulators, it briefly outlines our solution approach for *coordinated* manipulation, and it puts a focus on *cooperative manipulation*.

A. Framework Design

The *Dual-Arm Planner* is implemented in C++ and integrated in the higher-level software framework as outlined in Section III-B. To serve both *coordinated* and *cooperative* tasks with the same framework and to ensure a collision-free dual-arm manipulation requires a well-designed motion planning framework. Fig. 3 shows our implementation. It consists of three layers:

1) The first layer implements the ROS servers that reply to the requests from the *Task Planner*. For instance, we have a server that processes requests to assemble an object (*CoopAssembly*) or a server that moves both robots to specified goal configurations, i.e. *coordinated* manipulation (*CoordMove*).
2) The second layer implements the planning logic for servers that answer motion requests. It is implemented in the *Motion Framework* and will be outlined in the following.
3) The *Motion Framework* computes *Trajectories* that move the robots to defined goals or that enable an assembly process. These trajectories are forwarded to the third layer which consists of the two *Robot Controllers*.

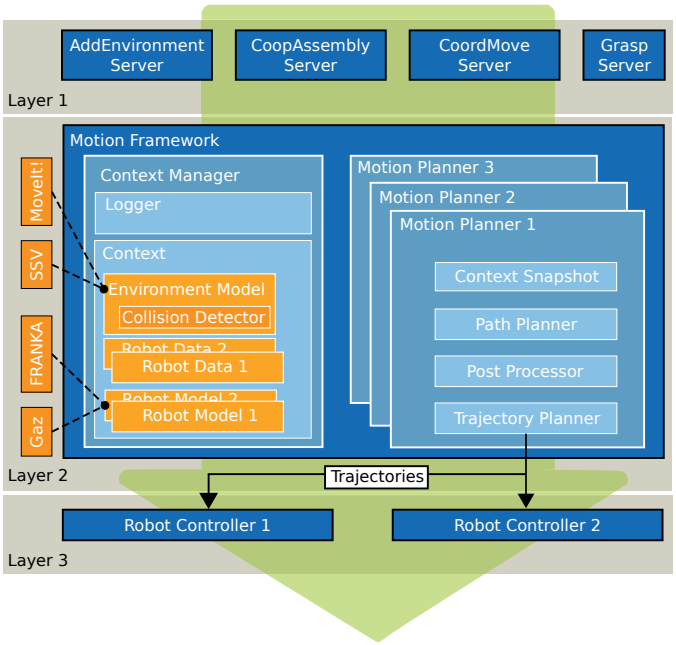


Fig. 3: Implementation of the *Dual-Arm Planner*.

As shown in Fig. 3, we implement multiple *Motion Planners* in the second layer that run in parallel in multiple threads, each with a different planning logic. This is because we need to serve different motion requests during the process: *Coordinated* manipulation assigns a desired goal configuration to each robot. This triggers collision-free motion planning in configuration space ($\mathscr{C}$-space). However, e.g. insertion tasks require a point-to-point motion of the end-effectors and thus planning has to be done in Cartesian space rather than $\mathscr{C}$-space. Further, when moving an object *cooperatively*, only the object's motion is computed and based on that the motions of the end-effectors are derived. Independent of the motion request type, the pipeline within a *Motion Planner* is the same, however the implemented logic differs:

1) A *Path Planner* computes collision-free waypoints between the current robot configuration and the desired goal. Planning collision-free motions is either performed using sampling-based planning approaches, e.g. *RRT* [19], or optimization-based approaches, e.g. *CHOMP* [20]. In our changing environment with two arms, optimization-based approaches suffer from local minima and we therefore rely on sampling-based approaches. For instance, for collision-free path planning in $\mathscr{C}$-space we use *RRTconnect* [21].
2) The sampling-based strategy leads to jerky paths and we therefore apply a *Post Processor*. We use state-of-the-art approaches like *Short-Cutting* [22] and we apply our proposed approach in [23] that minimizes distances between waypoints while keeping a clearance to obstacles.
3) The modified waypoints are forwarded to the *Trajectory Planner* that adds time information with cubic spline interpolation [24].
4) The trajectories are forwarded to the *Robot Controller*

Authorized licensed use limited to: Tsinghua University. Downloaded on March 08,2026 at 12:57:12 UTC from IEEE Xplore. Restrictions apply.
ISBN: 978-3-8007-6140-1

© VDE VERLAG GMBH · Berlin · Offenbach

TABLE II: Efficiency of the *Collision Detector* for 100.000 Queries.

	FCL	SSV
Total Time [s]	202.46	8.52
Avg. Time per Query [ms]	2.02	0.09

which might first adapt the trajectories: For instance, in *coordinated* manipulation, the motions of both robots might be computed independently of each other and therefore, each trajectory is collision-free but the combination of both is not, i.e. both robots collide during the execution. Thus, the two trajectories are coordinated such that also their simultaneous execution is collision-free.

Each of the three components of a *Motion Planner*, i.e. *Path Planner*, *Post Processor*, and *Trajectory Planner*, accesses the *Context*, our central knowledge database. It keeps an up-to-date *Environment Model* and stores the current robot states (*Robot Data*). Further, it includes the kinematic and dynamic *Robot Model*, e.g. to compute inverse kinematics or the current gravity vector.

We implement a *Collision Detector* in the *Environment Model* that allows for (self)collision checks. This is of major importance in dual-arm manipulation as collision checking often makes up 90 % of the computation time in motion planning [25]. We describe our collision computation approach in Section V-B.

To avoid damage to the hardware during testing and to enable quick development, we setup a *Gazebo* simulation of our mobile dual-arm system including dynamics information. Fig. 4 shows the mobile dual-arm robot both in real and in simulation, and it shows the collision model of the manipulators. We use the information in [26] to set up the dynamics information of our robots in the simulator.

B. Collision Detector

We model our robots and the environment with *Swept Sphere Volumes (SSV)* which are simplified geometries based on point, line and triangular elements. Fig. 4c shows the modeling of our robots and the environment based on this approach that we implemented in the *broccoli* library [27]. We benchmarked our modeling approach with the default collision detection approach that comes with *MoveIt!* [28], the state-of-the-art motion planning framework often used in literature. This default approach is based on the *Flexible Collision Library (FCL)* [29]. For each approach, we did 100.000 distance computations in the scenario in Fig. 4. For each distance computation, we set both robots to random configurations within their joint limits. Table II shows that, on average, with our *SSV* approach, we only take 4.5 % of the computation time that we would take with the *FCL* approach for one collision check. The control interface of our robots runs at 1 kHz. With our approach we can compute collisions online in real-time which we can not with the *FCL* approach as implemented in *MoveIt!*.

For offline collision checks, *MoveIt!* does not support collision-free motion planning for dual-arm/multiple robots in a shared workspace. This is still an open research topic and there are open projects within *MoveIt!* that aim to implement motion planning for multiple robot arms, e.g. [30].

C. Coordinated Manipulation

In this paper, we put a focus on *cooperative* manipulation and we only briefly outline our solution approach and current findings for *coordinated* manipulation. There are two different approaches for *coordinated* manipulation [31]:

- *Centralized* approaches combine the DoFs of both robots and consider the dual-arm setup as one single-arm setup with twice the amount of DoFs which in our case leads to a $\mathscr{C}$-space of dimension 14. Within this space, the state-of-the-art collision-free planning approaches for single-arm setups can be applied. However, in sampling-based path planning the exploration space grows exponentially with the $\mathscr{C}$-space dimension and so this approach is computationally expensive.
- *Decoupled* approaches handle some aspects of the planning independently for each robot. They are usually more efficient at the expense of sacrificing completeness. *Fixed-Path Coordination* [32] is a promising approach that has been successfully applied in industrial applications [33]. After planning collision-free trajectories for both robots independently, the dynamics of the trajectories are modified, so they do not collide. For instance, the velocity of one robot is decreased within a calculated time segment.

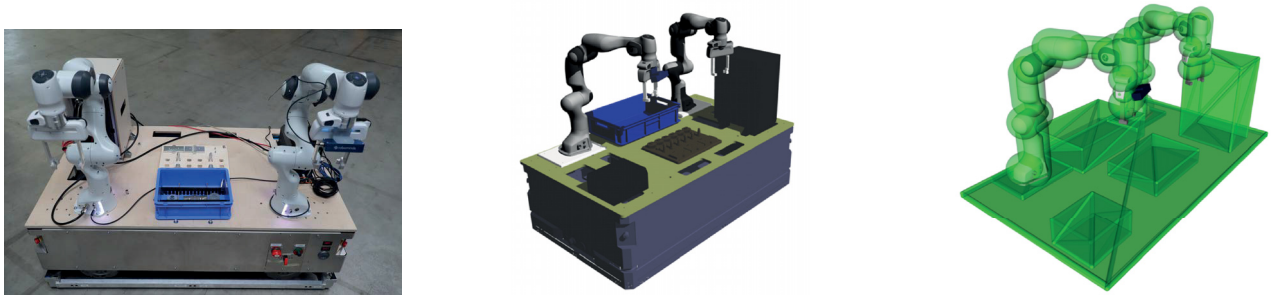
In preliminary experiments we have seen that, due to the efficient modeling presented in Section V-B, the difference in computation time between the *centralized* and *decoupled* approach is negligible (in the range of 100 ms). Therefore, within our multi-threaded architecture of Section V-A, we run the two approaches in parallel and take the first computed solution.

D. Cooperative Manipulation Control

The key challenge in *coordinated manipulation* is to avoid collisions at all time. The key challenge in *cooperative manipulation* is to control the internal forces that are induced by both robots manipulating the same object or assembling one object. We aim to rely only on joints torque measurements and further we want an approach that can be used for both tasks, i.e. manipulation and assembly. Finally, we require the compensation of the dynamic effects both of the manipulators and the mobile base. We assume that the manipulated objects are rigid and that their geometry and weight are known.

There are two steps in our cooperative manipulation: First, based on the current object pose in the world and the desired goal pose, we compute a point-to-point quintic spline trajectory for the object reference frame. Second, in online control, we derive the end-effector reference positions, velocities and accelerations, from the object trajectory, and compute the reference torques for both manipulators. We outline the steps in the following and we limit the notation to one robot, e.g.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 08,2026 at 12:57:12 UTC from IEEE Xplore. Restrictions apply.



(a) Developed mobile dual-arm robot hardware.

(b) Simulated mobile dual-arm robot in *Gazebo*.

(c) Collision model of the mobile dual-arm robot based on *SSV*.

Fig. 4: The algorithms developed for dual-arm manipulation are applied to the real hardware (Fig. 4a) and, without modification, to the simulated robots (Fig. 4b). Both use the same collision modeling (Fig. 4c).

we do not distinguish between the joint torques of the first robot τ_1 and second robot τ_2, but we limit the notation to τ. However, the computations are the same for both robots. Note that we use the provided joint velocity interface for *Coordinated Manipulation* and we use the joint torque interface for *Cooperative Manipulation*. That is, the control algorithms in Section V-C compute desired joint velocities and the algorithm in this section computes desired joint torques.

1) Grasp Kinematics: We denote the end-effector frame with B_p and the object frame, which is at the object's center of mass, with B_k. We assume that the objects are rigid and that their grasping poses are known, i.e. the homogeneous transformation between B_k and B_p. Based on the planned trajectory of the object frame B_k, we can derive the reference velocities $\dot{\mathbf{w}}_d$ and accelerations $\ddot{\mathbf{w}}_d$ of the end-effector frame B_p as follows:

$$\dot{\mathbf{w}}_d = \begin{pmatrix} \dot{\mathbf{r}}_p \\ \omega_p \end{pmatrix} = \underbrace{\begin{pmatrix} \mathbf{I}^{3\times3} & \tilde{\mathbf{r}}_{p\to k} \\ \mathbf{0}^{3\times3} & \mathbf{I}^{3\times3} \end{pmatrix}}_{\mathbf{G}^T} \begin{pmatrix} \dot{\mathbf{r}}_k \\ \omega_k \end{pmatrix} \tag{1}$$

$$\ddot{\mathbf{w}}_d = \begin{pmatrix} \ddot{\mathbf{r}}_p \\ \dot{\omega}_p \end{pmatrix} = \mathbf{G}^T \begin{pmatrix} \ddot{\mathbf{r}}_k \\ \dot{\omega}_k \end{pmatrix} - \begin{pmatrix} \tilde{\omega}_k \tilde{\omega}_k \mathbf{r}_{p\to k} \\ \mathbf{0}^{3\times1} \end{pmatrix} \tag{2}$$

Equation (1) is often proposed in literature and defines the *grasp matrix* $\mathbf{G}$. $\mathbf{G}^T$ maps the twist of B_k to B_p and $\dot{\mathbf{r}}$ and ω are the translational and rotational velocities. The formulation on acceleration level in (2) is our contribution. Therein, the tilde operator $\tilde{\star}$ defines the 3×3 skew-symmetric matrix operator performing the cross product.

2) Internal Object Force Estimation Based on Joint Torque Measurements: To control the induced internal object forces, as outlined in Section V-D3, we need to estimate these forces based on joint torque measurements τ_{sensor}. We assume that a dynamic model of the robot is given, i.e. the $\mathscr{C}$-space mass matrix $\mathbf{M}(\mathbf{q})$, the Coriolis and centrifugal forces terms $\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})$, the gravity torques $\mathbf{g}(\mathbf{q})$ and the friction $\mu(\dot{\mathbf{q}})$. For our *Franka Emika* robots, we use the model provided in [26] and we omit writing the dependencies on $\mathbf{q}$ and $\dot{\mathbf{q}}$ in the following. Further, we assume that the object mass m_{obj} and its moment of inertia Θ_{obj} w.r.t. the center of mass are known. Based on these dynamic models and our computed end-effector reference commands of Section V-D1, we estimate the internal object forces $\mathbf{F}_{int}$ as follows:

$$\dot{\mathbf{q}}_d = \mathbf{J}^{\#}\dot{\mathbf{w}}_d \quad \text{and} \quad \ddot{\mathbf{q}}_d = \mathbf{J}^{\#}\left(\ddot{\mathbf{w}}_d - \dot{\mathbf{J}}\dot{\mathbf{q}}\right) \tag{3}$$

$$\tau_{robot} = \mathbf{M}\ddot{\mathbf{q}}_d + \mathbf{C}\dot{\mathbf{q}}_d + \mathbf{g} + \mu \tag{4}$$

$$\mathbf{F}_{obj} = \begin{pmatrix} m_{obj}\mathbf{I}^{3\times3} & \mathbf{0}^{3\times3} \\ \mathbf{0}^{3\times3} & \Theta_{obj} \end{pmatrix} \begin{pmatrix} \ddot{\mathbf{r}}_k \\ \dot{\omega}_k \end{pmatrix} + \begin{pmatrix} -m_{obj}\mathbf{g} \\ \tilde{\omega}_k \Theta_{obj} \omega_k \end{pmatrix} \tag{5}$$

$$\mathbf{F}_{int} = \left(\mathbf{J}^T\right)^{\#}(\tau_{sensor} - \tau_{robot}) - \delta \cdot \mathbf{G}^{-1} \cdot \mathbf{F}_{obj} \tag{6}$$

Thereby, $\mathbf{J}$ is the end-effector Jacobian and $\mathbf{J}^{\#}$ is its pseudoinverse. Equation (6) computes the internal object forces $\mathbf{F}_{int}$ based on the joint torque measurements τ_{sensor}. The measured joint torques are a sum of the inertia forces of the robot arms due to their motion τ_{robot}, the inertia forces of the object due to its motion $\mathbf{F}_{obj}$, and the internal object forces due to the kinematic chain $\mathbf{F}_{int}$. We use the given dynamic robot model in (4) to compensate for the robot inertia and we use the object model proposed in [34] and given in (5) to compensate for the object inertia. The grasp matrix $\mathbf{G}$ in (6) transforms the object inertia forces from the center of mass to the end-effector frame and δ controls the load distribution between both manipulators: We set $\delta = 0.5$ to equally load both manipulators and we set $\Theta_{obj} = \mathbf{0}$ as we only weighed our manipulated objects but we did not determine their moment of inertia. That is, the object's angular momentum is treated as a disturbance. For our redundant manipulators, we derive the $\mathscr{C}$-space reference velocities $\dot{\mathbf{q}}_d$ and accelerations $\ddot{\mathbf{q}}_d$ with differential kinematics according to [35] and given in (3). Therein, the pseudoinverse of the Jacobian resolves the *under-determined* mapping between the given end-effector reference commands and the $\mathscr{C}$-space control commands by looking for the least squared norm solution of $\dot{\mathbf{q}}$. Equation (6), however, uses the pseudoinverse of the transposed Jacobian to resolve the *over-determined* mapping between the joint torques and the internal object forces acting at the end-effector. This ensures a least square error.

3) Cooperative Manipulation Control: Our cooperative manipulation control follows the *master-slave* concept [4]: We de-

Authorized licensed use limited to: Tsinghua University. Downloaded on March 08,2026 at 12:57:12 UTC from IEEE Xplore. Restrictions apply.

ISBN: 978-3-8007-6140-1 © VDE VERLAG GMBH · Berlin · Offenbach

fine a *master* robot that implements a stiff Cartesian impedance and that accurately tracks the computed trajectory:

$$\begin{aligned}\tau_{master} = \tau_{robot} &+ \mathbf{J}^T \delta \cdot \mathbf{G}^{-1} \cdot \mathbf{F}_{obj} \\ &+ \mathbf{J}^T \left(\mathbf{K}_p^p \mathbf{e}_p + \mathbf{K}_i^p \int \mathbf{e}_p + \mathbf{K}_d^p \dot{\mathbf{e}}_p \right) \\ &- \left(\mathbf{I} - \mathbf{J}^T \left(\mathbf{J}^{\#} \right)^T \right) k_n \dot{\mathbf{q}}\end{aligned} \tag{7}$$

Thereby, the compensation of the robots' and object's inertia is according to Section V-D2, and $\mathbf{e}_p$ is the error on position level between the desired trajectory and the current end-effector pose. Further, we add a k_n-weighted nullspace damping to limit nullspace motion.

The *master* robot is stiff and its objective is to track the desired motion. The *slave* robot's objective is to support the motion tracking and limit the internal object forces. Therefore, it implements the following control:

$$\begin{aligned}\tau_{slave} = \tau_{robot} &+ \mathbf{J}^T \delta \cdot \mathbf{G}^{-1} \cdot \mathbf{F}_{obj} \\ &+ \mathbf{J}^T \left(\mathbf{K}_p^p \mathbf{e}_p + (\mathbf{I} - \mathbf{S}) \mathbf{K}_i^p \int \mathbf{e}_p + \mathbf{K}_d^p \dot{\mathbf{e}}_p \right) \\ &- \left(\mathbf{I} - \mathbf{J}^T \left(\mathbf{J}^{\#} \right)^T \right) k_n \dot{\mathbf{q}} \\ &+ \mathbf{J}^T \mathbf{S} \left(\mathbf{K}_p^f \mathbf{e}_f + \mathbf{K}_i^f \int \mathbf{e}_f + \mathbf{K}_d^f \dot{\mathbf{e}}_f \right)\end{aligned} \tag{8}$$

Its compensation of inertia and its position control are similar to the master robot. However, the errors $\mathbf{e}_p$ and $\mathbf{e}_f$ now refer to the position and force error of the *slave* robot. An additional selection matrix $\mathbf{S}$ is added that defines directions in which we limit the internal object forces. For box handling, that we show in the accompanying video, this is the horizontal plane, i.e. the x- and y-directions. In these directions we apply an additional PID force control: Our desired internal force is $\mathbf{0}$ for cooperative manipulation and we compute the force error $\mathbf{e}_f$ with our internal force estimation of Section V-D2: $\mathbf{e}_f = \mathbf{0} - \mathbf{F}_{int}$. In the force-controlled directions, we only deactivate the integral term of the position controller by multiplying this term with $(\mathbf{I} - \mathbf{S})$ in (8), and we keep the proportional and derivative terms. Therefore, we refer to our approach as *hybrid-parallel position/force control*.

The implementation of our assembly controller is similar to the presented cooperative manipulation controller. The master robot keeps the first component at its current pose with position control. For insertion assembly, the slave robot pushes the second component against the first component with hybrid-parallel position/force control. Therefor, we define a desired internal object force, i.e. assembly force, unequal to $\mathbf{0}$. We add an additional wiggling motion to the master robot as presented in [36] to compensate for the robots' absolute inaccuracies. In the accompanying video we show an assembly operation.

VI. Navigator

This module is responsible for autonomously navigating the mobile base to storage and retrieval positions. For the autonomous navigation, we use the ROS framework *move_base* [37]. When *move_base* receives a goal, it first computes a path using its *global planner* based on the mobile base's static map. During execution of the path, the *local planner* adapts the trajectory based on the information provided by the *Lidars* to avoid dynamic obstacles that are not part of the map. The used *global planner* is the standard planner *navfn/NavfnROS*. As *local planner* we use *TebLocalPlannerROS*.

In addition to the autonomous navigation mode, we currently test a *deterministic path execution*. This includes the computation of the global plan by *move_base*. Subsequently, we compute the translational and rotational movements needed to execute the path analytically, which are executed regardless of dynamic obstacles. This alternative navigation mode allows the prediction of the mobile base's dynamic forces during path execution, which can then be compensated by control of the robot arms.

VII. Conclusion and Future Work

Our mobile dual-arm robot is a promising approach to increasing the efficiency of flexible production lines where AGVs are currently used exclusively for transport. Further, it has the potential to replace stationary preassembly work stations by taking over these kind of tasks. It is not limited to the presented gearbox assembly application and may well be applied to other production process with similar subtasks (transport, peg-in-hole insertion, box handling etc.). In addition, the algorithms developed are not limited to the hardware presented, e.g. the *Dual-Arm Planner* may be applied to other robots than the *Franka Emika* robot.

The accompanying video, which is also available at https://youtu.be/_aVbA6p1xZ0, shows the current state of our mobile robot as it navigates in a logistics facility and performs *coordinated* and *cooperative* manipulation.

Within the remaining project time we will investigate the dynamic effects between the mobile base and the control of the manipulators: Insertion and assembly operations require an accurate pose control of the end-effectors. However, during transport, the acceleration, and deceleration of the mobile base induce forces on the mounted manipulators. Their control must compensate for theses forces to ensure accurate assembly. Therefore, we consider to feedforward the expected forces and we consider to perform assembly only in arm configurations that show an advantageous manipulability w.r.t. the insertion direction: The *force transmission ratio* is a promising metric for finding arm configurations that are robust to the dynamic effects caused on the arms by the motion of the base [38].

Furthermore, we will extend the *Task Planner* by implementing recovery *Actions*. We will use them to react to errors that occur during the execution of the plan. For example, we will need *Actions* that are triggered in case the arms accidentally drop a part. The *Task Planner* should then attempt to locate and re-grasp the dropped part. If the part is outside the reachable work space of the robot arms, it will be removed from the current state and the goal state and the error will be logged.

References

[1] G. Ullrich and T. Albrecht, *Fahrerlose Transportsysteme: Eine Fibel - mit Praxisanwendungen - zur Technik - für die Planung*. Springer Fachmedien Wiesbaden, 2019.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 08,2026 at 12:57:12 UTC from IEEE Xplore. Restrictions apply.

[2] A. Coles, A. Coles, M. Fox, and D. Long, "Forward-chaining partial-order planning," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 20, 2010, pp. 42–49.

[3] M. Cashmore, M. Fox, D. Long, D. Magazzeni, B. Ridder, A. Carrera, N. Palomeras, N. Hurtos, and M. Carreras, "Rosplan: Planning in the robot operating system," in *Proceedings of the international conference on automated planning and scheduling*, vol. 25, 2015, pp. 333–341.

[4] F. Caccavale and M. Uchiyama, *Cooperative Manipulators*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 701–718.

[5] T. Wimböck and C. Ott, *Dual-Arm Manipulation*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 353–366.

[6] C. Smith, Y. Karayiannidis, L. Nalpantidis, X. Gratal, P. Qi, D. V. Dimarogonas, and D. Kragic, "Dual arm manipulation—a survey," *Robotics and Autonomous Systems*, vol. 60, no. 10, pp. 1340–1353, 2012.

[7] M. Tröbinger, C. Jähne, Z. Qu, J. Elsner, A. Reindl, S. Getz, T. Goll, B. Loinger, T. Loibl, C. Kugler, C. Calafell, M. Sabaghian, T. Ende, D. Wahrmann, S. Parusel, S. Haddadin, and S. Haddadin, "Introducing GARMI - A Service Robotics Platform to Support the Elderly at Home: Design Philosophy, System Overview and First Results," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5857–5864, 2021.

[8] C. Mayershofer, D.-M. Holm, B. Molter, and J. Fottner, "Loco: Logistics objects in context," in *2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2020, pp. 612–617.

[9] Bundesministerium für Bildung und Forschung, "ABB AG: Umhausungsfreie Montagestation," 2021.

[10] ABB, "Abb demonstrates concept of mobile laboratory robot for hospital of the future," 2019, https://new.abb.com/news/detail/37301/abb-demonstrates-concept-of-mobile-laboratory-robot-for-hospital-of-the-future, visited 2023-03-03.

[11] C. Lawrence, "Rise and fall of rethink robotics," 2019, https://www.asme.org/topics-resources/content/rise-fall-of-rethink-robotics, visited 2023-03-03.

[12] Kawasaki Robotics, "duARO - | Doppelarm Scara Roboter von Kawasaki Robotics," 2015.

[13] Yaskawa Europe GmbH, "SDA-Serie CSDA10F," 2021.

[14] KAWADA Robotics Corporation, "NEXTAGE Works Side by Side With People," 2015.

[15] T. Asfour, M. Waechter, L. Kaul, S. Rader, P. Weiner, S. Ottenhaus, R. Grimm, Y. Zhou, M. Grotz, and F. Paus, "Armar-6: A high-performance humanoid for human-robot collaboration in real-world scenarios," *IEEE Robotics Automation Magazine*, vol. 26, no. 4, pp. 108–121, 2019.

[16] M. Fuchs, C. Borst, P. Robuffo Giordano, A. Baumann, E. Kraemer, J. Langwald, R. Gruber, N. Seitz, G. Plank, K. Kunze, R. Burger, F. Schmidt, T. Wimboeck, and G. Hirzinger, "Rollin' justin - design considerations and realization of a mobile platform for a humanoid upper body," in *2009 IEEE International Conference on Robotics and Automation*, 2009, pp. 4131–4137.

[17] SICK GmbH, "Flexi Soft CPU," 2023, https://www.sick.com/de/de/sicherheitssteuerungen/sicherheitssteuerungen/flexi-soft/c/g186176, visited 2023-03-07.

[18] robominds GmbH, "robobrain.vision 2.0: schneller, intelligenter, industrialisierter," 2023, https://www.robominds.de/produkte/robobrain, visited 2023-03-03.

[19] S. M. LaValle, "Rapidly-exploring random trees : a new tool for path planning," *The annual research report*, 1998.

[20] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "Chomp: Gradient optimization techniques for efficient motion planning," in *2009 IEEE International Conference on Robotics and Automation*, 2009, pp. 489–494.

[21] J. Kuffner and S. LaValle, "Rrt-connect: An efficient approach to single-query path planning," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, vol. 2, 2000, pp. 995–1001 vol.2.

[22] R. Geraerts and M. H. Overmars, "Creating high-quality paths for motion planning," *The International Journal of Robotics Research*, vol. 26, no. 8, pp. 845–863, 2007.

[23] J. Wittmann, C. Klein, and D. Rixen, "Path quality improvement of sampling-based planners: An efficient optimization-based approach using analytical gradients," in *ROMANSY 24 - Robot Design, Dynamics and Control*. Springer, 2022, pp. 182–190.

[24] J. Wittmann and D. J. Rixen, "Time-optimization of trajectories using zero-clamped cubic splines and their analytical gradients," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4528–4534, 2022.

[25] M. Elbanhawi and M. Simic, "Sampling-based robot motion planning: A review," *IEEE Access*, vol. 2, pp. 56–77, 2014.

[26] C. Gaz, M. Cognetti, A. Oliva, P. Robuffo Giordano, and A. De Luca, "Dynamic Identification of the Franka Emika Panda Robot With Retrieval of Feasible Parameters Using Penalty-Based Optimization," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4147–4154, 2019.

[27] P. Seiwald and F. Sygulla, "broccoli: Beautiful Robot C++ Code Library," 2022, https://gitlab.lrz.de/AM/broccoli.

[28] D. Coleman, I. A. Sucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: a moveit! case study," *ArXiv*, vol. abs/1404.3785, 2014.

[29] J. Pan, S. Chitta, and D. Manocha, "Fcl: A general purpose library for collision and proximity queries," in *2012 IEEE International Conference on Robotics and Automation*, 2012, pp. 3859–3866.

[30] PickNik Robotics, "Tutorial for multiple robot arms," 2022, https://moveit.ros.org/events/2022-google-summer-of-code/#tutorial-for-multiple-robot-arms, visited 2023-03-03.

[31] L. E. Kavraki and S. M. LaValle, *Motion Planning*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 109–131.

[32] T. Simeon, S. Leroy, and J.-P. Lauumond, "Path coordination for multiple mobile robots: a resolution-complete algorithm," *IEEE Transactions on Robotics and Automation*, vol. 18, no. 1, pp. 42–49, 2002.

[33] F. Beuke, S. Alatartsev, S. Jessen, C. Hanel, and A. Verl, "Online motion planning for dual-arm industrial robots," in *ISR 2018; 50th International Symposium on Robotics*, 2018, pp. 1–8.

[34] M. Unseren, "Rigid body dynamics and decoupled control architecture for two strongly interacting manipulators," *Robotica*, vol. 9, no. 4, p. 421–430, 1991.

[35] D. Whitney, "Resolved motion rate control of manipulators and human prostheses," *IEEE Transactions on Man Machine Systems*, vol. 10, no. 2, pp. 47–53, 1969.

[36] J. Wittmann, F. Pachler, P. Ruhkamp, H. Jung, F. Sygulla, and D. Rixen, "Robotic framework for autonomous assembly: a report from the robothon 2021 grand challenge," in *2022 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, 2022, pp. 59–65.

[37] E. Marder-Eppstein, D. Lu, M. Ferguson, and A. Hoy, "Move_base—ros wiki," *[online] Available: http://wiki. roseorg/move_base*, 2016.

[38] S. Chiu, "Control of redundant manipulators for task compatibility," in *Proceedings. 1987 IEEE International Conference on Robotics and Automation*, vol. 4, 1987, pp. 1718–1724.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 08,2026 at 12:57:12 UTC from IEEE Xplore. Restrictions apply.